Chapter 22 worked solutions

Object Detection and Segmentation

Exercises 1–4 work through box and mask overlap, assignment during training, and ranked detections. Exercises 9–11 use these ideas to design evaluations.

1. Compare two boxes

Call the first box $A = (1, 1, 4, 3)$ and the second $B = (2, 0, 5, 2)$. The four numbers are the left, top, right, and bottom edges. In continuous corner coordinates, subtract the left edge from the right edge to get the width, and the top edge from the bottom edge to get the height. Each box has width 3 and height 2, so both have area $3 \cdot 2 = 6$ square units.

To find the shared rectangle, start at the later of the two left edges and stop at the earlier of the two right edges. The shared horizontal span runs from $x = 2$ to $x = 4$, so it has width 2. The shared vertical span runs from $y = 1$ to $y = 2$, so it has height 1. The overlap area is $2 \cdot 1 = 2$. Adding the box areas counts that overlap twice, so subtract it once to get a union area of $6 + 6 - 2 = 10$. Intersection over union (IoU) is the overlap area divided by the union area, giving $2/10 = 1/5$.

Moving B one unit right adds 1 to both of its x coordinates, so the moved box is $B' = (3, 0, 6, 2)$. Each box still has area 6. The overlap now runs from $x = 3$ to $x = 4$ and from $y = 1$ to $y = 2$. Its area is $1 \cdot 1 = 1$, and the union area is $6 + 6 - 1 = 11$. The new IoU is $1/11$.

2. Count shared pixels

Draw rows from top to bottom and columns from left to right, starting at zero in both directions. For example, $(1, 2)$ is in the middle row and the rightmost column, so that cell is marked in mask B. A 1 denotes a foreground pixel and a 0 denotes a background pixel.

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \qquad B = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

The intersection marks a cell when both masks contain a 1. The union marks a cell when either mask contains a 1, including cells where both do.

$$A \cap B = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \qquad A \cup B = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

The two shared pixels are $(0, 1)$ and $(1, 1)$, so the **intersection count is 2**. The union grid contains two marked cells in the first row, two in the second, and one in the third. Its **union count is 5**. Equivalently, the two masks mark 3 and 4 pixels, and subtracting the 2 shared pixels gives $3 + 4 - 2 = 5$.

3. Make a unique assignment

Call the objects A and B and the queries Q1, Q2, and Q3. Q1 is the cheapest query for A at cost 2 and for B at cost 1. Choosing the minimum in each column independently would use Q1 for both objects and give an apparent total of 3. A one-to-one assignment uses a different query for each object.

Choose a query for A, then try each of the two queries left for B. This lists all six valid assignments.

Query for A	Query for B	Total cost
Q1	Q2	$2 + 5 = 7$
Q1	Q3	$2 + 2 = 4$
Q2	Q1	$3 + 1 = 4$
Q2	Q3	$3 + 2 = 5$
Q3	Q1	$4 + 1 = 5$
Q3	Q2	$4 + 5 = 9$

The minimum cost is **4**, achieved by two assignments. Assigning Q1 to A and Q3 to B leaves Q2 unmatched. Assigning Q2 to A and Q1 to B leaves Q3 unmatched. Each assignment gives the two objects distinct queries. If the training matcher selects either solution, its unmatched query receives the no-object target.

4. Track precision and recall

Start with the highest-scoring candidate. It is a false positive, so after one prediction there are zero matches. The second candidate is a true positive, so the count rises to one. At that point one of the two predictions is correct, giving precision $1/2$, and one of the three annotations has been matched, giving recall $1/3$.

At each rank, divide the number of true positives seen so far by the number of candidates seen to get precision. Divide the same true-positive count by the total of three annotated objects to get recall.

Rank	Outcome	True positives so far	Precision	Recall
1	False positive	0	$0/1 = 0$	$0/3 = 0$
2	True positive	1	$1/2$	$1/3$
3	True positive	2	$2/3$	$2/3$
4	False positive	2	$2/4 = 1/2$	$2/3$

The maximum recall is $2/3$, first reached at rank 3. The final candidate lowers precision because it adds a prediction without adding a match. Even keeping all four candidates leaves one of the three annotations unmatched, so full recall would require a prediction that matches the remaining object.

9. Compare two prompting protocols (optional)

Suppose one image contains two annotated buses. A promptable segmenter can trace the mask of the first bus if we give it the annotated box. In use, the detector may supply a box that cuts off the front of that bus, and it may miss the second bus entirely. The two prompt protocols let us measure these effects separately.

Use the same held-out images and ground-truth instance masks in both protocols. For each annotated instance, record the image ID, instance ID, class, and reference mask. With annotated boxes, send the instance's own box as its only prompt. With detector boxes, freeze the detector and its confidence cutoff using validation data. Associate predictions to annotations for scoring in descending confidence order, within the same image and class, using each annotation at most once. For example, accept the highest-overlap unused annotation when box IoU is at least 0.5. If two boxes compete for the same bus, only the first qualifying match is paired with that bus. Record unmatched detections and unmatched annotations separately.

Run the same segmentation checkpoint, preprocessing, box coordinate conversion, and output rule in both protocols. If the segmenter returns several masks, choose the one with its highest predicted mask-quality score, using a fixed tie rule. Compute the chosen mask's IoU against the reference mask on the original image grid. The quality score is the model's estimate, whereas measured IoU uses the annotation. In the detector-box protocol, generate boxes without consulting the evaluation annotations. In both protocols, choose the output mask without looking at the reference mask. The evaluator uses the annotations to pair and score the results.

Report annotated-box mask IoU over all annotated instances. Then report the IoU of both protocols on the **same subset** of instances with an accepted detection. Pair the two mask scores by instance ID and report the mean of detector-box IoU minus annotated-box IoU. That subset comparison shows what changes when the prompt box changes, among instances the detector found. The full annotated-box protocol measures segmentation given known object identity and location. It bypasses finding the object and can also reveal a poor mask even when a good box is supplied.

Show the count and fraction of annotations with no matched detector box. Report an all-instance mean mask IoU with missed detections scored zero. For example, if a detector finds one of the two buses and its mask scores 0.8, the detected-only mean is 0.8, the missed-detection rate is $1/2$, and the mean with misses assigned zero is $0.8/2 = 0.4$. The zero records that the system returned no mask for that instance. Count extra detections as false positives separately because averaging over reference instances alone does not penalize them.

10. Audit resized annotations (optional)

Imagine that a small image is doubled in size and placed inside a larger padded canvas. A detector returns boxes in canvas coordinates. The left edge of the image content is now inside the canvas, so we must remove that displacement before undoing the scale. For masks, we must also remove pixels in the padded border.

Record the original width and height, the actual resized width and height, and the left and top padding in pixels. Also record any right and bottom padding so the content region can be recovered. Convert predicted boxes from normalized coordinates or center-size form to corner coordinates on the padded canvas if necessary. If x' and y' are coordinates on the canvas, subtract the corresponding padding and then divide by the scale on that axis.

$$x = \frac{x' - p_{left}}{s_x}, \qquad y = \frac{y' - p_{top}}{s_y}.$$

Apply this rule to both corners, using the actual ratios of resized to original dimensions for s_x and s_y. Clip boxes to the original bounds if the scoring protocol requires it and check that any retained box has positive area. Compute box IoU on that original coordinate system. Rounding edge coordinates to integers before the comparison can move a small box by a noticeable fraction of its width.

For a mask, first map its grid to the padded canvas using the checkpoint's specified postprocessing. Extract exactly the resized image region by removing the padded border, then resample to the original height and width. For an already Boolean mask, use nearest-neighbor resampling to retain discrete labels. If the model returns mask logits or probabilities, use its specified interpolation and apply the fixed foreground threshold at the documented stage. Check that the predicted and reference masks have the same original grid shape before comparing foreground pixels. When constructing a box prompt in the other direction, multiply by the scale and then add the padding.

A tiny check starts with width 4 and height 3. Resize to width 8 and height 6, then pad one column on the left and two rows on top. The original box $(1, 0, 3, 2)$ maps to $(3, 2, 7, 6)$ on the canvas. Removing the offsets and dividing by two recovers $(1, 0, 3, 2)$, which gives IoU 1 against the original box. Dividing by two alone produces $(1.5, 1, 3.5, 3)$ and gives IoU 3/13. The shifted result stays inside the original bounds, so a bounds check alone would miss the mistake.

As a companion mask check, mark just pixel $(1, 2)$ in original (row, column) coordinates. Doubling each dimension gives a 2×2 foreground block. On the padded canvas it occupies rows 4–5 and columns 5–6. Correctly removing the padding and using nearest-neighbor resampling returns the single foreground pixel at $(1, 2)$. Compare the full expected grid as well as the shape. A crop taken from the wrong origin can still produce an array with the expected dimensions.

11. Separate detection errors (optional)

Draw a road scene as one row of four objects. The bracketed letters mark the four annotations, with the corresponding boxes given below. The image is 21 pixels wide and 6 pixels high. All box coordinates use the chapter's left, top, right, bottom edge convention.

```
left edge    [ A: car ]   [ B: bus ]   [ C: bicycle ]   [ D: bus ]
```

Use an evaluation IoU threshold of 0.5, with one-to-one matching within each class. Keep predictions with confidence at least 0.4 for the initial audit. The detector produces the following candidates.

Object and true box	Predictions near that object	Error to inspect
A, $(1, 1, 4, 4)$	Car $(1, 1, 4, 4)$ at 0.95 and car $(2, 1, 5, 4)$ at 0.45	Duplicate
B, $(6, 1, 9, 4)$	Truck $(6, 1, 9, 4)$ at 0.82	Wrong class
C, $(12, 1, 15, 4)$	Bicycle $(10, 1, 13, 4)$ at 0.76	Misplaced box
D, $(17, 1, 20, 4)$	No candidate at any confidence	Missing instance

For A, both predictions have the right label. The exact box claims A first. The second box has intersection area 6 and union area 12, for IoU 0.5. It would qualify on overlap, but A is already used, so the second detection is a duplicate false positive. For B, the truck has IoU 1 with the bus box and still fails to match the bus because the class differs. That gives a truck false positive and leaves B unmatched. For C, the bicycle box has intersection area 3 and union area 15, so IoU is 0.2. It has the right label and fails the localization threshold, so the prediction is a false positive and C remains unmatched. For D, there is no box to score against the annotation.

Raising the confidence cutoff from 0.4 to 0.5 removes the lower scoring duplicate and retains the correct car detection. In a larger set, raising that cutoff can also remove valid detections with low confidence. Here it cannot change the truck to a bus or move the bicycle box onto the object. A higher cutoff could hide those false positives, but both objects would still lack correct detections. A lower cutoff can recover a missing instance when a suitable candidate was scored below the cutoff. D has no candidate in this example. Correcting these three errors would require a correct class, a better box, and a new detection, respectively.